

WebGPU

Moderne GPU-Berechnung für das Web

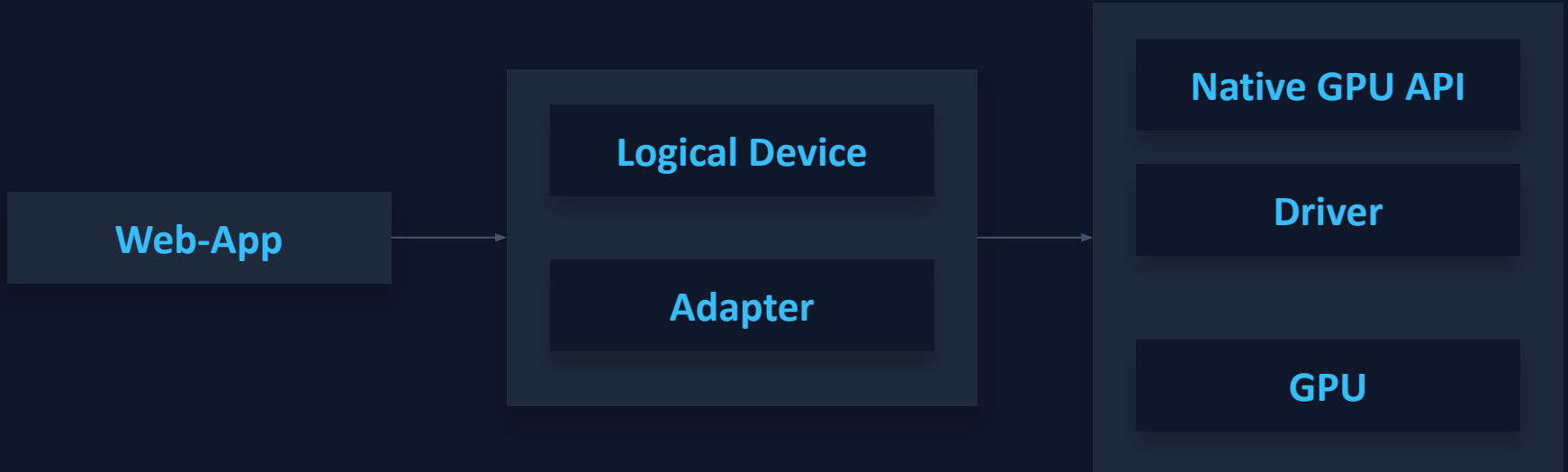
Historie

- Nachfolger von WebGL
- WebGL -> OpenGL
- WebGL war weniger performant
- Keine Unterstützung von Metal, Vulkan, Direct3D 12
- Konnte GPGPU nicht gut handeln

Was ist WebGPU?

- W3C-Standard-API für GPU-Zugriff übers Web
- Abbildung auf Vulkan, Metal, Direct3D 12 auf nativer Ebene
- Unterstützt Grafik-Rendering und Allzweck-Compute (GPGPU)

Kern-Architektur



Pipeline-Typen

Render-Pipeline

- Vertex → Rasterisierung → Fragment → Ausgabe

Compute-Pipeline

- Compute Stage

Vertex-Shader Beispiel

```
const vertexShaderCode = `
// Vertex Position als Input
struct VertexInput {
    @location(0) position: vec2<f32>,
    @location(1) color: vec3<f32>,
};

// Output an Fragment Shader
struct VertexOutput {
    @builtin(position) position: vec4<f32>,
    @location(0) color: vec3<f32>,
};

// @vertex markiert dies als Vertex Shader
@vertex
fn main(input: VertexInput) -> VertexOutput {
    var output: VertexOutput;
    // Position im Clip-Space (von -1 bis 1)
    output.position = vec4<f32>(input.position, 0.0, 1.0);
    // Farbe weitergeben
    output.color = input.color;
    return output;
}
`;
```

```
const vertices = new Float32Array([
    // Vertex 1: Oben, rot
    0.0, 1, 1.0, 0.0, 0.0,
    // Vertex 2: Links unten, grün
    -1, -1, 0.0, 1.0, 0.0,
    // Vertex 3: Rechts unten, blau
    1, -1, 0.0, 0.0, 1.0,
]);
```

Fragment-Shader Beispiel

```
// Schritt 6: Fragment Shader definieren
// Fragment Shader bestimmt die Farbe jedes Pixels
const fragmentShaderCode = `
    // Input vom Vertex Shader
    struct FragmentInput {
        |   @location(0) color: vec3<f32>,
    };

    // @fragment markiert dies als Fragment Shader
    @fragment
    fn main(input: FragmentInput) -> @location(0) vec4<f32> {
        |   // Gib die interpolierte Farbe zurück
        |   return vec4<f32>(input.color, 1.0);
    }
`;
```

Fragen?

Quellen

W3C Spec: [w3.org/TR/webgpu](https://www.w3.org/TR/webgpu)

https://developer.mozilla.org/en-US/docs/Web/API/WebGPU_API